

Guia de Laboratorio
Cluster Nginx Aplicaciones L7
v1

I. Introducción:

Cuando se está trabajando en aplicaciones que serán objeto de grandes cantidades de peticiones por segundo, es necesario disponer de una solución de infraestructura que permita soportar, escalar y mantener en operación dichas aplicaciones. Para ello se dispone de una solución de balanceo de carga a través de Nginx, el cual integrando nodos backend nos crean una solución cluster para poder operar de manera eficiente en este escenario.

II. Resultados de aprendizaje:

- Desplegar un balanceador de carga L7 a través de Nginx.
- Atender las consultas provenientes de un balanceador de carga.
- Desplegar una aplicación de L7 a través de un entorno de archivos de acceso múltiple.

III. Arquitectura de Despliegue

Para desplegar un servicio de Balanceador L7, se debe considerar que un equipo desplegado con Nginx hará la tarea de balancear todo el tráfico con destino al puerto 80/443 hacia los servidores reales, como se muestra en la figura no. 1. Note que Balanceador-1, está en modo activo operando para realizar el balanceo de carga, y Balanceador-2 está en modo standby en espera de que si falla el Balanceador activo, entrará en operación de forma automática. Esto implica que deberá valerse del protocolo heartbeat o pacman para establecer una IP flotante que permita designar a uno u otro balanceador de carga.

El balanceo de carga recibe las peticiones en el puerto 80/443 y las redirige a través de la directiva **proxy_pass** hacia los nodos que tendrán a su vez la configuración de las aplicaciones que se quieren desplegar. Tendrá que tomar en consideración que este es un despliegue de laboratorio, y para ponerlo a producción en un ambiente real, necesitará de un servicio de persistencia de sesiones como **Memcache** o **Redis** para que la aplicación sea operativa en los aspectos donde los usuarios requieran autenticación.

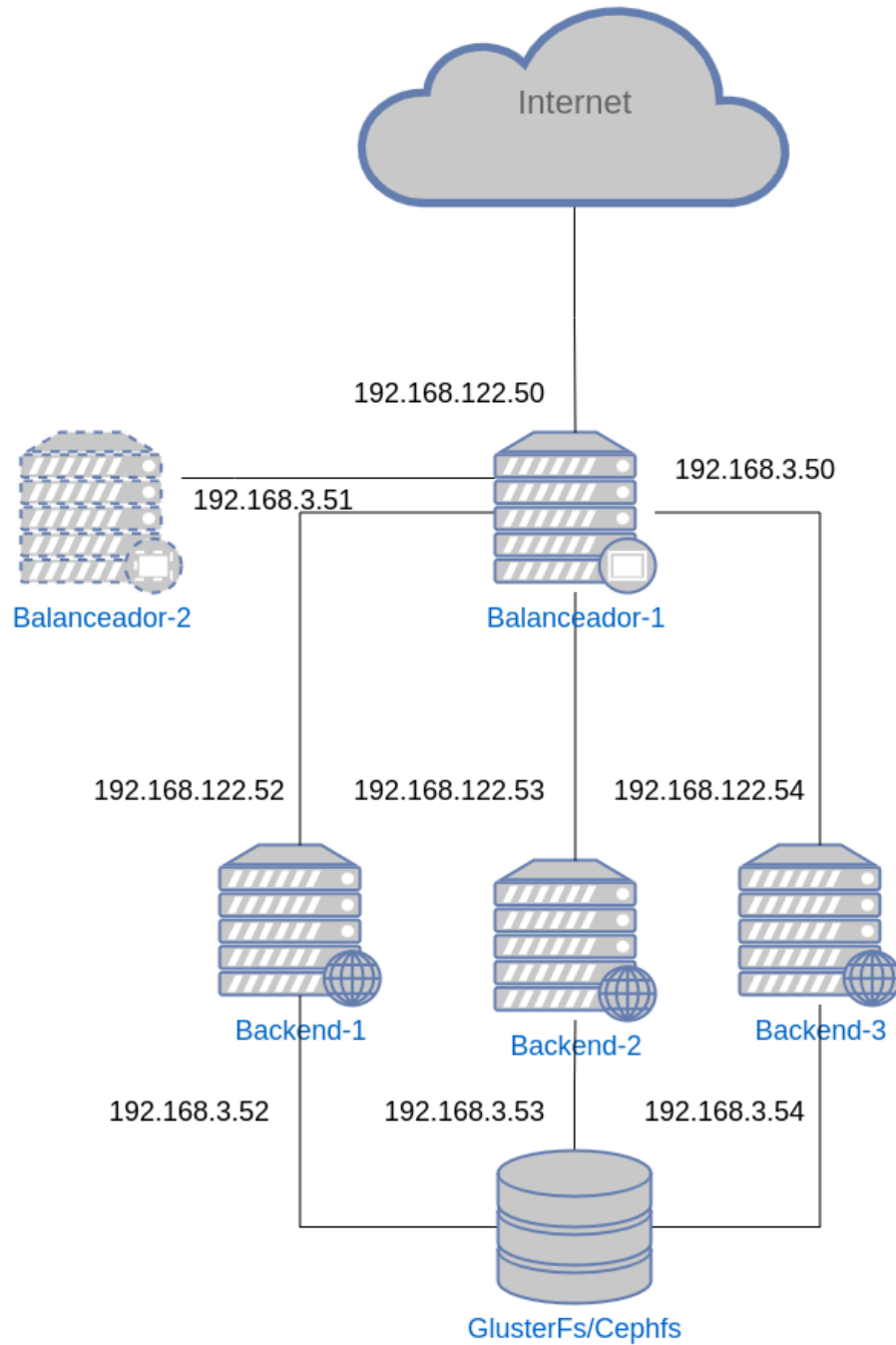


Figura No. 1 Topología de Balanceador L7

IV. Instalación

Para llevar a cabo el proceso de instalación, deberá seguir los siguientes pasos, en todos los equipos que participen del cluster:

```
$sudo apt install nginx
```

Debe realizar esta instalación tanto en el balanceador como en los nodos.

Debe considerar que una vez instalado el paquete este crea una estructura de archivos a en la carpeta `/etc/`, sobre la cual se realizan las configuraciones.

Directorio/Archivo	Descripción
<code>/etc/nginx</code>	Directorio principal del paquete para fines de configuración
<code>/etc/nginx/sites-available</code>	Directorio con las configuraciones de los diferentes Virtual Host configurados en el servicio.
<code>/etc/nginx/sites-enabled</code>	Directorio con las configuraciones de los diferentes Virtual Host configurados y puestos a producción con el servicio.
<code>/etc/nginx/nginx.conf</code>	Archivo principal de configuración del servicio

V. Configuración

Como podrá suponer la configuración de los nodos responderá a las mismas condiciones, para el caso del balanceador de carga, la configuración estará en función de crear un **upstream** el cual tiene la función de definir los servidores que participarán del cluster de balanceo y los cuales serán alcanzados por el balanceador para el enrutamiento de la carga. Deberá considerar que el puerto destino de los servidores puede ser diverso, se recomienda siempre utilizar puertos asociados al protocolo http, para no interferir con otros servicios.

Es importante destacar que cuando se requiere que las aplicaciones utilicen SSL para conexiones seguras, no es necesario que los nodos dispongan de dicha configuración, sino simplemente el balanceador de carga. Es decir tendrá que definirse la configuración SSL solo a nivel del balanceador y establecer los certificados necesarios para su funcionamiento.

Estrategias de Balanceo Nginx

round-robin	Se hace una distribución de las peticiones de los clientes hacia los servidores de manera cíclica, de tal manera de mantener con la misma carga a todos los nodos.
least-connected	Las peticiones son enviadas a los nodos que poseen menos conexiones activas. Se deberá considerar que el monitoreo de las conexiones requiere consumo de recursos.
ip-hash	En este modo el balanceador establece de manera aleatoria en el enrutamiento hacia el nodo destino, y mantiene adherida a él las conexiones siguientes siempre que el socket origen no cambie. Si el socket origen cambia, se repite el ciclo de asignación.

Para nuestro caso se estará realizando la configuración más generalizada que es round-robin.

5.1. Configuración del Balanceador

A continuación se detalla la configuración que se establecerá en el archivo de configuración para el Nodo Balanceador.

/etc/nginx/sites-available/default

```
upstream cluster {
    server nodo1:80;
    server nodo2:80;
    server nodo3:80;
}
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name _;

    location / {

        proxy_pass http://cluster;
        add_header X-Upstream $upstream_addr;
    }
}
```

Note que el archivo de configuración requiere que se establezca un upstream para definir los nodos que participarán del cluster, y posteriormente hacer el envío hacia el proxy_pass. Si no se define una forma de ruteo, el servicio opta por el default **round-robin**

Para este caso como en la directiva **server** se están utilizando nombres, deberá considerar incluirlos en el archivo de configuración **/etc/hosts**

Puede retormar aspectos más avanzados de configuración del balanceador, como el paso de la IP real de los host clientes apoyados en la guía de Nginx:

<https://docs.nginx.com/nginx/admin-guide/load-balancer/using-proxy-protocol/>

5.2. Configuración de los Nodos

Los nodos deberán tener una configuración estándar, a través del puerto que se pondrá a disposición del servicio, por default 80. Para ellos deberá considerar como ejemplo de configuración el archivo que se muestra a continuación.

Deberá tomar en cuenta que para optar por algún servicio de intérprete de lenguaje de programación, la configuración podría ser un poco diferente.

/etc/nginx/sites-available/default

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    root /var/www/html;

    index index.html index.htm index.nginx-debian.html;

    server_name _;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Para este caso, se está poniendo a disposición del servicio el puerto escuchando en todas las direcciones IP, además se está estableciendo el directorio `/var/www/html` como directorio raíz para alojar los archivos de aplicación web.

Para fines de laboratorio, considere modificar el archivo index en cada uno de los nodos, de tal manera que permita saber cuando esté realizando las pruebas en cual nodo está enviando las peticiones.

VI. Desplegando y configurando Redis Server

Redis brinda un servicio de caché para el almacenamiento de información en memoria y que pueda ser accesible desde cualquier otro dispositivo a través de la red a altas velocidades. Puede ser utilizada además con un repositorio de sesiones de usuario establecidas creadas a partir de algún lenguaje de programación, de tal manera que las aplicaciones dispuestas en un Cluster L7 puedan acceder a ellas siempre y cuando el usuario se encuentre autenticado en uno de los nodos. Lo cual permite desplegar aplicables escalables como aspectos desacoplados.

El servicio será desplegado utilizando el **puerto TCP 6379**

6.1 Instalación de Redis

La instalación de redis es parte de los repositorios debian y ubuntu, por lo tanto podrá instalarlo a través de la siguiente orden.

```
$sudo apt install redis-server
```

Una vez instalado deberá establecer algunos parámetros de configuración para que el servidor sea operativo. Entre ellos habilitar el socket de conexión para que pueda ser accedido a través de las direcciones que posee el servidor, ya que por default, lo pone disponible para el

loopback. Revisar el archivo de configuración especificado en la ruta posterior y verificar que las directivas estén configuradas según se detalla.

```
/etc/redis/redis.conf
```

```
bind 192.168.3.250
requirepass icc115
```

Una vez realizada esta configuración con los aspectos mínimos, deberá reiniciar el servicio para que los cambios tengan efecto.

```
$sudo /etc/init.d/redis-server restart
```

6.1.1. Redis Cluster

Puede configurar un redis cluster para llevar una copia de cada uno de los objetos que se guarden en un servidor redis. La configuración mínima son Master/Slave, por lo tanto, si usted desea hacer una replicación de cada uno de los servidores, en el caso de tener 3 máster, deberá tener también 3 slave. Para lo cual necesitaría 6 servidores.

En el caso de hacer una replica simple usted podrá hacerlo de la siguiente manera:

```
#redis-cli --cluster create 192.168.122.170:6379
192.168.122.171:6379 192.168.122.172:6379 -a icc115
```

Si desea hacerlo con réplica 1, deberá de ejecutar:

```
#redis-cli --cluster create 192.168.122.170:6379
192.168.122.171:6379 192.168.122.172:6379 192.168.122.173:6379
192.168.122.174:6379 192.168.122.175:6379 --cluster-replicas 1 -a
icc115
```

Deberá de tomar en cuenta que el modo cluster debe estar activo en cada uno de los nodos en el archivo de configuración `/etc/redis/redis.conf`

```
cluster-enabled yes
```

6.2. Probando el servicio

Para probar el servicio, podrá hacerlo desde el mismo o servidor a través de un equipo remoto, podrá utilizar el modo sin autenticación y posteriormente probar el modo autenticado.

```
damian@redis:/home/damian# redis-cli -h 192.168.122.250
```

```
192.168.122.250:6379> keys *
(error) NOAUTH Authentication required.
192.168.122.250:6379> AUTH icc115
OK
192.168.122.250:6379> exit
```

6.3 Configurando el servicio para ser utilizado por PHP

Aunque existen muchas maneras de consumir el servicio de redis, en el caso de PHP posee una solución muy práctica que se configura en el archivo `php.ini` de la versión de php instalado.

Para PHP deberá instalar la extensión necesaria para poder desplegar el servicio, esto para el caso del Cluster L7 solo deberá hacerlo en los Nodos, no es necesario hacerlo en el Balanceador.

```
$sudo apt install php-redis
```

Para nuestro caso, asumimos que se ha instalado la versión `php-fpm` ya que el laboratorio se ha estado elaborando con `nginx`. En dicho caso la configuración deberá hacerse en la ruta siguiente:

`/etc/php/8.1/fpm/php.ini`

```
session.save_handler = "tcp://192.168.3.250:6379?auth=icc115,
tcp://192.168.3.251:6379?auth=icc115"
```

Cuando abra el archivo deberá buscar la directiva `session.save_handler` para poder establecer la ruta hacia el servidor redes, así como los parámetros de autenticación establecidos en el mismo. Esta directiva en PHP establece la ubicación de las sesiones que se crean cuando se accede a una aplicación web.

Deberá crear un nuevo Virtual Host en `nginx` para brindar la configuración que permita desplegar PHP-FPM, se recomienda hacer un archivo con el nombre de la Aplicación a Desplegar.

`/etc/nginx/sites-available/wordpress`

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    root /var/www/html;
    location / {
        try_files $uri /index.php$is_args$args;
    }

    location ~ ^/index\.php(/|$) {
        fastcgi_pass unix:/var/run/php/php8.1-fpm.sock;
        fastcgi_split_path_info ^(.+\.php)(/.*)$;
        include fastcgi_params;
    }
}
```

```
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
        fastcgi_param DOCUMENT_ROOT $realpath_root;
        internal;
    }

    error_log /var/log/nginx/wordpress_error.log;
    access_log /var/log/nginx/wordpress_access.log;
}
```

Para llevarlo a producción deberá hacer un enlace simbólico de esta configuración:

```
$sudo ln -s /etc/nginx/sites-available/wordpress /etc/nginx/sites-enabled/
```

Deberá reiniciar el servicio para que los cambios tengan efecto.

```
$sudo /etc/init.d/php8.1-fpm restart
$sudo /etc/init.d/nginx restart
```

Una vez realizado esto estará listo para poder desplegar los servicios de autenticación haciendo uso de PHP, y que las sesiones sean guardadas en Redis para que puedan ser leídas por todos los nodos.

VII. Requerimientos de Laboratorio

Crear un Cluster L7 con alto rendimiento y alta disponibilidad que permita desplegar una aplicación de Wordpress, valiéndose de los componentes desplegados con anterioridad: Cluster de BD, Cluster de Disco. Así como los aspectos tratados en este laboratorio. Se le pide desplegar la topología siguiente:

